ECE 526L Spring 2025 Lab 8

Sum of Products Circuit — Hierarchical Design and Testing



Muhammad Umar
CSUN ID: 204533244

Instructor: Prof. Orod Haghighiara

I hereby attest that this lab report is entirely my own work. I have not copied either code or text from anyone, nor have I allowed or will I allow anyone to copy my work.

Name (printed): **Muhammad Umar**

Name (signed):  Date: 04/23/2025

INTRODUCTION

In this lab, a hierarchical Sum of Products (**SOP**) circuit was designed and verified using Verilog HDL. The circuit implements a digital filtering operation by multiplying data inputs with corresponding coefficients and summing the results. Following the lab specification, the design was built with three levels of hierarchy: the first level included parameterized **register**, **multiplier**, and **adder** modules; the second level grouped these into an SOP block; and the top level combined two SOP blocks and a final adder.
Special emphasis was placed on scalable and reusable design through parameterization, proper signal sizing to prevent overflow, and structured verification practices. Both non-exhaustive and randomized exhaustive testbenches were developed to thoroughly validate the functionality. Control structures like **$monitoron**, **$monitoroff**, and forced **error injection** using compiler directives ensured that the test strategy was efficient, thorough, and compliant with best digital design practices.

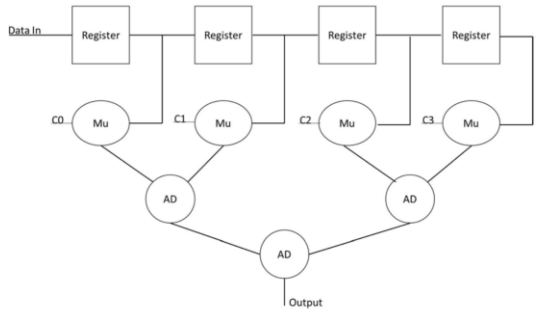


Figure 8.1 Block Diagram of SOP Circuit

METHODOLOGY AND TEST PLAN

First of all we have to build Register, Multiplier and Adder module, These are the core design modules which have to be used for calculation of the sum of product.

1. Register Module (register.v)

This module implements a parameterizable register with synchronous reset functionality. It stores the input **d** into output **q** on every **rising edge** of the clock (clk).

The design uses an always **@(posedge clk)** block, which ensures that data updates only on a clock edge. Including a **synchronous reset** improves simulation control, allowing the register to start in a known state.
It also ensures safe behavior during hardware initialization, avoiding unpredictable or undefined register contents at power-up or during reconfiguration. This makes the circuit more reliable, robust, and easier to verify through automated testbenches.

2. Adder Module (adder.v)

This module implements a scalable, parameterized adder.
It takes two WIDTH-bit inputs a and b and produces a (WIDTH+1)-bit sum output to handle potential overflow.

a. **a (input):** First operand.
b. **b (input):** Second operand.
c. **sum (output):** Result of a + b, widened by one bit.

We used a continuous assignment (assign) statement for efficient synthesis and straightforward behavior.

3. Multiplier Module (multiplier.v)

This module implements a scalable, parameterized multiplier.
It multiplies two WIDTH-bit operands and produces a (**2*WIDTH**)-bit product **p**.

a. **a (input):** First operand.
b. **b (input):** Second operand.
c. **p (output):** Product of a and b, double the input width.

The module uses a continuous assignment statement to perform unsigned multiplication between the operands.

4. Sum of Products Block (sop_block.v)

This module builds a second-level block of the Sum of Products (SOP) circuit. It consists of **two registers**, **two multipliers**, and **one adder**. The **clk** (clock) input synchronizes the register operations, while the **rst** (reset) input clears the registers when asserted which means it is assumed as active high. The primary inputs **data0** and **data1** are registered, and then each registered output is multiplied by its corresponding coefficient inputs **coef0** and **coef1**. The two resulting products are then summed using an adder to produce the output **sum_out**. To keep track of accurate results and prevent data loss, the width of the signals is properly expanded at each stage, following the lab's requirement for safe arithmetic operations.

5. Top-Level Sum of Products (sop_top.v)

This module represents the highest level of hierarchy for the Sum of Products (SOP) circuit. It combines two instances of the sop_block module and a final adder to produce the overall output. The clk (clock) input synchronizes the entire system, while the rst (reset) input controls the clearing of internal registers. Inputs **data0** through **data3** serve as the primary data signals, and **coef0** through **coef3** provide the corresponding coefficients. The outputs of the two SOP blocks are summed by a final adder, resulting in the **sop_result**, which represents the complete Sum of Products computation.

Structure:

a. The first sop_block handles **data0, data1, coef0, coef1**.
b. The second sop_block handles **data2, data3, coef2, coef3**.
c. Their outputs are summed with a final adder to produce **sop_result**.

Width adjustments are made at each level to maintain data integrity during addition.

6. Non-Exhaustive-Testbench (sop_top_tb_non_exhaustive.v)

The non-exhaustive testbench verifies the functionality of the sop_top module using a small set of chosen input vectors. It applies manually selected test cases designed to cover a wide range of input conditions, including zero inputs, maximum data values, bit patterns, symmetric patterns, ascending and descending sequences, and alternating

high/low patterns. Each test case verifies that the sop_result output matches a manually calculated expected value. The specific test cases include all zero inputs with all zero coefficients, mixed increasing and decreasing values, maximum data values with minimal coefficients, activation of a single data-coefficient pair, bit pattern shifting, alternating maximum and zero values, uniform mid-range values across all inputs, crisscross patterns between data and coefficients, ascending and descending value patterns, and matching progressive inputs for both data and coefficients.

TABLE 1. Non-Exhastive Test Cases

Test #	Data Inputs	Coef Inputs	Description
1	0, 0, 0, 0	0, 0, 0, 0	All zero case
2	1, 2, 3, 4	4, 3, 2, 1	Mixed values
3	15,15,15,15	1, 1, 1, 1	Max data values
4	1, 0, 0, 0	15, 0, 0, 0	Single pair active
5	8, 4, 2, 1	1, 2, 4, 8	Bit pattern test
6	15, 0, 15, 0	0, 15, 0, 15	Alternating high/low
7	7, 7, 7, 7	7, 7, 7, 7	Uniform mid-range values
8	5, 10, 5, 10	10, 5, 10, 5	Crisscross pattern
9	9, 6, 3, 0	0, 3, 6, 9	Ascending/descending pattern
10	1, 2, 3, 4	1, 2, 3, 4	Matching progressive inputs

A non-exhaustive testbench tests a small, carefully selected set of input vectors to quickly check basic functionality. However, non-exhaustive testing **cannot guarantee** that every possible input combination has been covered. It may miss subtle bugs that occur under specific rare conditions.

In contrast, an **exhaustive testbench** systematically or randomly applies a **very large number of different input combinations** to the design. This method dramatically increases the confidence that the design is fully correct, even under unexpected scenarios. Exhaustive testing ensures that corner cases, edge cases, and hidden functional issues are exposed, which is critical for validating hardware designs intended for real-world use where any input pattern could occur.

Thus, while a non-exhaustive test is fast and helpful for initial debugging, an exhaustive test is necessary for **complete functional verification**.

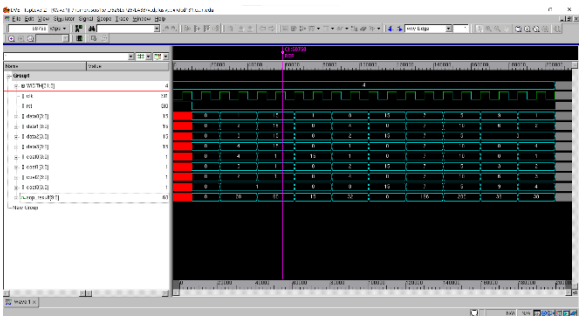


Figure 8.2 Waveform of Non-Exhaustive Test Cases

7. Exhaustive-Testbench (sop_top_tb_exhaustive.v)

The exhaustive testbench was developed to thoroughly verify the functionality of the hierarchical Sum of Products (SOP) design. In this testbench, all primary inputs data0, data1, data2, data3, and coef0, coef1, coef2, coef3 — were independently randomized to cover a broad range of input combinations.

A total of 1,000,000 random test vectors were applied to the circuit. For each transaction, the expected output was dynamically calculated by performing the same sum of

products operation used in the design, ensuring a direct comparison between the DUT output and the reference result. If a mismatch was detected, the simulation was immediately stopped, and the inputs, expected result, and actual output were displayed for debugging purposes.

To manage simulation output, **$monitoron** and **$monitoroff** commands were used to show only the first dozen and last dozen transaction results. Additionally, a forced error was introduced after approximately 75% of the total vectors using the **FORCE_ERROR** compiler directive to confirm that the testbench would correctly detect injected faults.

If all four data inputs and all four coefficient inputs were allowed to vary independently over their 4-bit range (0–15), the total number of possible input combinations would be:

$$16^8 = 4{,}294{,}967{,}296 \text{ cases}$$

Simulating all 4.3 billion combinations exhaustively would be impractical and extremely time-consuming, potentially taking days to complete even on a fast simulation environment. Therefore, randomized exhaustive testing was used instead, providing high coverage while keeping the simulation time manageable.

This approach significantly improves confidence in the circuit's correctness by exposing the design to a wide and randomized variety of input scenarios, without the impractical overhead of a full deterministic exhaustive run.

a) Use of $monitoron and $monitoroff:

To manage the simulation output as required in lab manual, **$monitoron** was enabled during the first 12 and the last 12 transactions to log critical test data. Between these periods, **$monitoroff** was used to suppress intermediate printing, keeping the simulation logs clean, concise, and manageable. This strategy ensured that both the beginning and end portions of the simulation were properly recorded for verification purposes, while avoiding excessive and redundant output during the bulk of randomized testing.

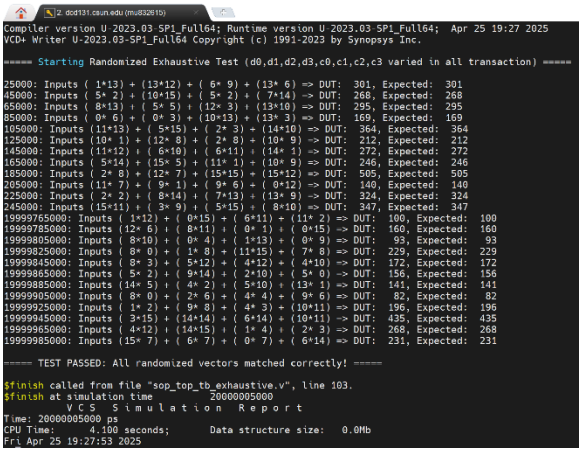


Figure 8.3 Log of First and Last Test Cases

b) Error Free Waveform of Exhaustive Test:

The error-free waveform of the exhaustive test shows correct input stimulus and corresponding output results throughout the simulation. No mismatches were observed, confirming that the DUT behaved as expected for all randomized input vectors.

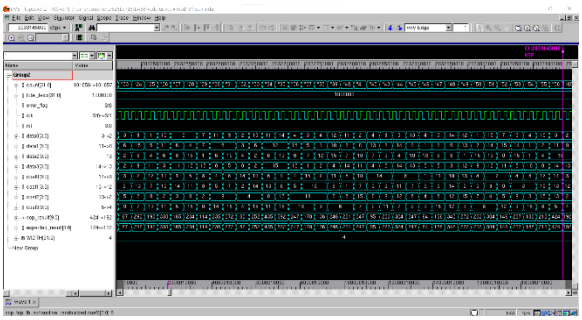


Figure 8.4 Waveform of Error Free Exhaustive Testbench

c) Error Injected Waveform of Exhaustive Test:

The error-injected waveform of the exhaustive test captures the moment when a forced mismatch was introduced after approximately 75% of the transactions. A behavioral reference model was implemented within the testbench to dynamically compute the expected result, and the DUT output was continuously compared against this reference to detect any mismatches, When the mismatch happens the test should be stopped, We have created an error in a sense that just adds 1 in reference model so that it should mimic an error.

```
Transaction Passed at time 14999845000  Expected:  133, Got:  133
Transaction Passed at time 14999865000  Expected:  467, Got:  467
Transaction Passed at time 14999885000  Expected:  251, Got:  251
Transaction Passed at time 14999905000  Expected:  137, Got:  137
Transaction Passed at time 14999925000  Expected:  108, Got:  108
Transaction Passed at time 14999945000  Expected:  156, Got:  156
Transaction Passed at time 14999965000  Expected:  149, Got:  149
Transaction Passed at time 14999985000  Expected:  242, Got:  242
Transaction Passed at time 15000005000  Expected:  222, Got:  222

ERROR at time 15000025000: Mismatch Detected!
Inputs: ( 5* 9) + ( 8* 8) + ( 4* 8) + ( 3*11)
Expected:  175, Got:  174
$finish called from file "sop_top_tb_exhaustive.v", line 91.
$finish at simulation time         15000045000
           V C S   S i m u l a t i o n   R e p o r t
Time: 15000045000 ps
CPU Time:      6.590 seconds;       Data structure size:   0.0Mb
Sat Apr 26 19:26:24 2025
$
```

Figure 8.5 Log of Force Error Exhaustive Testbench

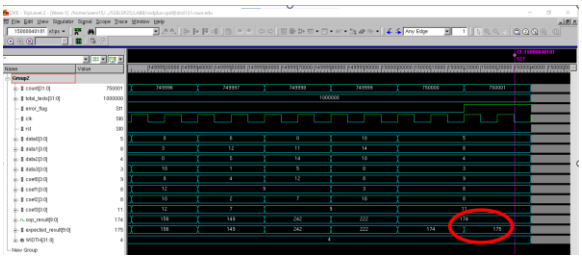


Figure 8.6 Log of Force Error Exhaustive Testbench

EXTRA CREDIT:

Reset Decision

A reset input (**rst**) was included in the register design. Resetting ensures the system starts from a known state, eliminates undefined behavior during simulation startup, and enhances the testbench control. This design decision aligns with best digital design practices, even though the original specification did not mandate it.

RESULTS:

In the non-exhaustive testing, a carefully selected set of ten input vectors was applied to verify basic functionality, covering key cases such as all-zero inputs, maximum values, single activation, bit patterns, and symmetric sequences. All transactions passed successfully, with the DUT output matching the physical counted expected results.

For the exhaustive randomized testing, 1,000,000 randomly generated input combinations were applied, varying all four data and coefficient inputs independently. A non-hierarchical reference model was implemented within the testbench to calculate the expected output dynamically for each transaction. Throughout the error-free simulation, all DUT outputs matched the expected outputs, confirming correct

functionality. During the forced error test, a mismatch was correctly detected after approximately 75% of the transactions, demonstrating that the testbench was able to identify errors as designed.

Waveform captures were taken to verify the correct operation at the start and end of the error-free run, and around the moment the forced error was injected.

CONCLUSION

The hierarchical Sum of Products (SOP) circuit was successfully designed, implemented, and verified through both non-exhaustive and exhaustive randomized testing. The use of scalable modules for the register, multiplier, and adder ensured flexibility and proper width management throughout the design. Verification strategies, including a behavioral reference model and dynamic output comparison, provided high confidence in the correctness of the design. All **non-exhaustive** vectors and randomized **exhaustive** vectors passed without errors during normal simulation. A forced error was correctly detected.

This lab emphasized the importance of structured hierarchy, parameterization, exhaustive verification, and intelligent use of simulation controls like **$monitoron**, **$monitoroff**, and conditional forced errors. The project successfully met all design and verification objectives, ensuring a high degree of reliability in the final hardware model.

APPENDIX

Appendix A: adder.v

```
/*************************************************************************
*** EE 526 L Experiment #8            Muhammad Umar, Spring, 2025     ***
***                                                                   ***
*** Experiment #8 Sum of Products Circuit Hierarchical Design and     ***
*** Testing.                                                          ***
***                                                                   ***
*** Filename: adder.v                                                 ***
***                          Created by Muhammad Umar, 04/20/2025     ***
***                                                                   ***
*** This module implements a parameterized adder that sums two        ***
*** WIDTH-bit inputs and outputs a (WIDTH+1) bit result to handle     ***
*** overflow.                                                         ***
*************************************************************************/
`timescale 1ns/1ps

module adder #(parameter WIDTH = 4) (
    input wire [WIDTH-1:0] a,
    input wire [WIDTH-1:0] b,
    output wire [WIDTH:0] sum
);
    assign sum = a + b;
endmodule
```

Appendix B: multiplier.v

```
/*************************************************************************
*** EE 526 L Experiment #8            Muhammad Umar, Spring, 2025     ***
***                                                                   ***
*** Experiment #8 Sum of Products Circuit Hierarchical Design and     ***
*** Testing.                                                          ***
***                                                                   ***
*** Filename: multiplier.v                                            ***
***                          Created by Muhammad Umar, 04/20/2025     ***
***                                                                   ***
*** This module implements a parameterized multiplier that computes   ***
*** the product of two WIDTH-bit inputs, producing a (2*WIDTH)-bit    ***
*** output.                                                           ***
*************************************************************************/

`timescale 1ns/1ps

module multiplier #(parameter WIDTH = 4) (
    input wire [WIDTH-1:0] a,
    input wire [WIDTH-1:0] b,
    output wire [(2*WIDTH)-1:0] p
);
    assign p = a * b;
endmodule
```

Appendix C: register.v

```
/*************************************************************************
*** EE 526 L Experiment #8            Muhammad Umar, Spring, 2025     ***
***                                                                   ***
*** Experiment #8 Sum of Products Circuit Hierarchical Design and     ***
*** Testing.                                                          ***
***                                                                   ***
*** Filename: register.v                                              ***
***                          Created by Muhammad Umar, 04/20/2025     ***
***                                                                   ***
*** This module implements a parameterized register with synchronous  ***
*** reset. It stores a WIDTH-bit input on the rising clock edge and   ***
*** clears the output when reset is asserted.                         ***
*************************************************************************/

`timescale 1ns/1ps

module register #(parameter WIDTH = 4) (
    input wire clk,
    input wire rst,
    input wire [WIDTH-1:0] d,
    output reg [WIDTH-1:0] q
);
    always @(posedge clk) begin
        if (rst)
            q <= 0;
        else
            q <= d;
    end
endmodule
```

Appendix D: sop_block.v

```
/*************************************************************************
*** EE 526 L Experiment #8            Muhammad Umar, Spring, 2025     ***
***                                                                   ***
*** Experiment #8 Sum of Products Circuit Hierarchical Design and     ***
*** Testing.                                                          ***
***                                                                   ***
*** Filename: sop_block.v                                             ***
***                         Created by Muhammad Umar, 04/20/2025      ***
***                                                                   ***
*** This module implements a hierarchical Sum of Products block.      ***
*** It multiplies two pairs of data and coefficients, sums the        ***
*** products, and outputs a (2*WIDTH)-bit result.                     ***
*************************************************************************/

`timescale 1ns/1ps

module sop_block #(parameter WIDTH = 4)(
    input wire clk,
    input wire rst,
    input wire [WIDTH-1:0] data0, data1,
    input wire [WIDTH-1:0] coef0, coef1,
    output wire [(2*WIDTH):0] sum_out
);
    wire [WIDTH-1:0] reg_out0, reg_out1;
    wire [(2*WIDTH)-1:0] prod0, prod1;
    wire [(2*WIDTH):0] adder_out;

    // Registers
    register #(WIDTH) r0 (.clk(clk), .rst(rst), .d(data0),
.q(reg_out0));
    register #(WIDTH) r1 (.clk(clk), .rst(rst), .d(data1),
.q(reg_out1));

    // Multipliers
    multiplier #(WIDTH) m0 (.a(reg_out0), .b(coef0), .p(prod0));
    multiplier #(WIDTH) m1 (.a(reg_out1), .b(coef1), .p(prod1));

    // Adder (width = 2*WIDTH)
    adder #(2*WIDTH) a0 (.a(prod0), .b(prod1), .sum(adder_out));

    assign sum_out = adder_out;
endmodule
```

Appendix E: sop_top.v

```
/*************************************************************************
*** EE 526 L Experiment #8            Muhammad Umar, Spring, 2025     ***
***                                                                   ***
*** Experiment #8 Sum of Products Circuit Hierarchical Design and     ***
*** Testing.                                                          ***
***                                                                   ***
*** Filename: sop_top.v                                               ***
***                          Created by Muhammad Umar, 04/20/2025     ***
***                                                                   ***
*** This module implements the top-level Sum of Products circuit.     ***
*** It sums the outputs of two SOP blocks and produces a              ***
*** (2*WIDTH+1)-bit final result.                                     ***
*************************************************************************/

`timescale 1ns/1ps

module sop_top #(parameter WIDTH = 4)(
    input wire clk,
    input wire rst,
    input wire [WIDTH-1:0] data0, data1, data2, data3,
    input wire [WIDTH-1:0] coef0, coef1, coef2, coef3,
    output wire [(2*WIDTH+1):0] sop_result
);
    wire [(2*WIDTH):0] sum0, sum1;
    wire [(2*WIDTH+1):0] final_sum;

    // Two SOP blocks
    sop_block #(WIDTH) sb0 (
        .clk(clk), .rst(rst),
        .data0(data0), .data1(data1),
        .coef0(coef0), .coef1(coef1),
        .sum_out(sum0)
    );

    sop_block #(WIDTH) sb1 (
        .clk(clk), .rst(rst),
        .data0(data2), .data1(data3),
        .coef0(coef2), .coef1(coef3),
        .sum_out(sum1)
    );

    // Final adder
    adder #(2*WIDTH+1) final_adder (
        .a(sum0), .b(sum1), .sum(final_sum)
```

```
    );

    assign sop_result = final_sum;
endmodule
```

Appendix F: sop_top_tb_exhaustive_randomized.v:

```
/*************************************************************************
*** EE 526 L Experiment #8            Muhammad Umar, Spring, 2025     ***
***                                                                   ***
*** Experiment #8 Sum of Products Circuit Hierarchical Design and     ***
*** Testing.                                                          ***
***                                                                   ***
*** Filename: sop_top_tb_exhaustive_randomized.v                      ***
***                         Created by Muhammad Umar, 04/20/2025      ***
***                                                                   ***
*** This testbench verifies the SOP design by applying 1,000,000      ***
*** randomized input combinations and comparing the DUT output        ***
*** against a reference model.                                        ***
*************************************************************************/

`timescale 1ns/1ps

module sop_top_tb_exhaustive_randomized;

    parameter WIDTH = 4;
    integer count = 0;
    integer total_tests;
    reg error_flag;  // Error flag to track mismatch

    reg clk;
    reg rst;
    reg [WIDTH-1:0] data0, data1, data2, data3;
    reg [WIDTH-1:0] coef0, coef1, coef2, coef3;
    wire [(2*WIDTH+1):0] sop_result;

    // Expected reference output
    reg [(2*WIDTH+1):0] expected_result;

    // Instantiate the DUT
    sop_top #(WIDTH) DUT (
        .clk(clk), .rst(rst),
        .data0(data0), .data1(data1),
        .data2(data2), .data3(data3),
        .coef0(coef0), .coef1(coef1),
        .coef2(coef2), .coef3(coef3),
        .sop_result(sop_result)
    );

    // Clock generation
    always #5 clk = ~clk;
```

```
    initial begin
        clk = 0;
        rst = 1;
        #10 rst = 0;

        $vcdpluson;

        total_tests = 1000000;  // Randomize 100,0000 test vectors
        error_flag = 0;

        $display("\n===== Starting Randomized Exhaustive Test
(d0,d1,d2,d3,c0,c1,c2,c3 varied in all transaction) =====\n");

        $monitor("%0t: Inputs (%d*%d) + (%d*%d) + (%d*%d) + (%d*%d)
=> DUT: %d, Expected: %d",
                 $time,
                 data0, coef0, data1, coef1, data2, coef2, data3,
coef3,
                 sop_result, expected_result);

        $monitoroff;

        repeat (total_tests) begin
            // Randomize inputs
            data0 = $random % 16;
            data1 = $random % 16;
            data2 = $random % 16;
       data3 = $random % 16;
            coef0 = $random % 16;
            coef1 = $random % 16;
            coef2 = $random % 16;
            coef3 = $random % 16;

       expected_result = (data0*coef0) + (data1*coef1) + (data2*coef2)
+ (data3*coef3);

            @(posedge clk);
            @(posedge clk);



            count = count + 1;
```

```
            `ifdef FORCE_ERROR
            if (count > (total_tests * 3) / 4)
                expected_result = expected_result + 1;  // Force
error injection
            `endif

            // Control monitor output
            if (count <= 12 || count >= (total_tests - 12))
                $monitoron;
            else
                $monitoroff;

            // Compare
            if (sop_result !== expected_result) begin
                $display("\nERROR at time %0t: Mismatch Detected!",
$time);
                $display("Inputs: (%d*%d) + (%d*%d) + (%d*%d) +
(%d*%d)",
                         data0, coef0, data1, coef1, data2, coef2,
data3, coef3);
                $display("Expected: %d, Got: %d", expected_result,
sop_result);
                error_flag = 1;
        @(posedge clk);
        @(posedge clk);
                $finish;
            end
        else begin
                $display("Transaction Passed at time %0t  Expected:
%d, Got: %d", $time, expected_result, sop_result);
            end
        end

        // Final result
        if (error_flag == 0) begin
            $display("\n===== TEST PASSED: All randomized vectors
matched correctly! =====\n");
        end
        else begin
            $display("\n===== TEST FAILED: Errors detected during
simulation. =====\n");
        end

        $finish;
```

```
    end

endmodule
```

Appendix G: sop_top_tb_non_exhaustive

```
/*************************************************************************
*** EE 526 L Experiment #8            Muhammad Umar, Spring, 2025     ***
***                                                                   ***
*** Experiment #8 Sum of Products Circuit Hierarchical Design and     ***
*** Testing.                                                          ***
***                                                                   ***
*** Filename: sop_top_tb_non_exhaustive.v                             ***
***                         Created by Muhammad Umar, 04/20/2025      ***
***                                                                   ***
*** This testbench applies a small set of carefully selected input    ***
*** vectors to verify the basic functionality of the hierarchical SOP ***
*** design.                                                           ***
*************************************************************************/

`timescale 1ns/1ps

module sop_top_tb_non_exhaustive;

    parameter WIDTH = 4;

    reg clk;
    reg rst;
    reg [WIDTH-1:0] data0, data1, data2, data3;
    reg [WIDTH-1:0] coef0, coef1, coef2, coef3;
    wire [(2*WIDTH+1):0] sop_result;

    // DUT instantiation
    sop_top #(WIDTH) DUT (
        .clk(clk),
        .rst(rst),
        .data0(data0), .data1(data1),
        .data2(data2), .data3(data3),
        .coef0(coef0), .coef1(coef1),
        .coef2(coef2), .coef3(coef3),
        .sop_result(sop_result)
    );

    // Clock generation: 10ns period
    always #5 clk = ~clk;

    initial begin
        $vcdpluson;
        clk = 0;
        rst = 1;
```

```
        #10 rst = 0;

        $display("\n================== Starting Non-Exhaustive SOP
Test ==================\n");
   $display("============================================================
====================");
        $display("Time\tData Inputs\t\tCoef Inputs\tSOP Result");
   $display("============================================================
====================\n\n");

        // Test cases with clear context
        apply_inputs(0, 0, 0, 0, 0, 0, 0, 0, "All zero input check");
        apply_inputs(1, 2, 3, 4, 4, 3, 2, 1, "Mix of
increasing/decreasing values");
        apply_inputs(15, 15, 15, 15, 1, 1, 1, 1, "Max data input
test");
        apply_inputs(1, 0, 0, 0, 15, 0, 0, 0, "Single active
multiplier path");
        apply_inputs(8, 4, 2, 1, 1, 2, 4, 8, "Bit pattern test
(shifting weights)");
        apply_inputs(15, 0, 15, 0, 0, 15, 0, 15, "Alternating pattern
test");
        apply_inputs(7, 7, 7, 7, 7, 7, 7, 7, "Uniform mid-range
values");
        apply_inputs(5, 10, 5, 10, 10, 5, 10, 5, "Crisscross mix
pattern");
        apply_inputs(9, 6, 3, 0, 0, 3, 6, 9, "Descending and
ascending combination");
        apply_inputs(1, 2, 3, 4, 1, 2, 3, 4, "Same sequence on both
inputs");

        $display("\n================================== Non Exhausted
Test Completed ================================== \n");
        $finish;
    end

    // Stimulus task with message
    task apply_inputs;
        input [WIDTH-1:0] d0, d1, d2, d3;
        input [WIDTH-1:0] c0, c1, c2, c3;
        input [255:0] message;

        begin
```

```
$display("============================================================
==============");
            $display("Test: %0s", message);

$display("============================================================
==============");

            data0 = d0; data1 = d1; data2 = d2; data3 = d3;
            coef0 = c0; coef1 = c1; coef2 = c2; coef3 = c3;

            @(posedge clk);  // Register data
            @(posedge clk);  // Wait for result to stabilize

            $display("%0t\t%d %d %d %d\t\t%d %d %d %d\t= %d\n",
                $time,
                data0, data1, data2, data3,
                coef0, coef1, coef2, coef3,
                sop_result
            );
        end
    endtask

endmodule
```